Evaluation	1	3
Algorithms	2	9
Society, Ethics, and the Profession	Included in SEP hours	
Total	**9**	**26**

Knowledge Units

PDC-Programs: Programs

CS Core:

1. Parallelism
    a. Declarative parallelism: Determining which actions may, or must not, be performed in parallel, at the level of instructions, functions, closures, composite actions, sessions, tasks, and services is the main idea underlying PDC algorithms; failing to do so is the main source of errors. (See also: PDC-Algorithms)
    b. Defining order: for example, using happens-before relations or series/parallel directed acyclic graphs representing programs.
    c. Independence: determining when ordering does not matter, in terms of commutativity, dependencies, preconditions.
    d. Ensuring ordering among otherwise parallel actions when necessary, including locking, safe publication; and imposing communication – sending a message happens before receiving it; conversely relaxing when unnecessary.
2. Distribution
    a. Defining places, as devices executing actions, including hardware components, remote hosts, may also include external, uncontrolled devices, hosts, and users. (See also: AR-IO)
    b. One device may time-slice or otherwise emulate multiple parallel actions by fewer processors by scheduling and virtualization. (See also: OS-Scheduling)
    c. Naming or identifying places (e.g., device IDs) and actions as parties (e.g., thread IDs).
    d. Activities across places may communicate across media. (See also: PDC-Communication)
3. Starting activities
    a. Options that enable actions to be performed (eventually) at places range from hardwiring to configuration scripts; also establishing communication and resource management; these are expressed differently across languages and contexts, usually relying on automated provisioning and management by platforms (See also: SF-Resources)
    b. Procedural: Enabling multiple actions to start at a given program point; for example, starting new threads, possibly scoping, or otherwise organizing them in hierarchical groups
    c. Reactive: Enabling upon an event by installing an event handler, with less control of when actions begin or end, and may apply even on uniprocessors
    d. Dependent: Enabling upon completion of others; for example, sequencing sets of parallel actions (See also: PDC-Coordination)
    e. Granularity: Execution cost of action bodies should outweigh the overhead of arranging them

4. Execution Properties
    a. Nondeterministic execution of unordered actions
    b. Consistency: Ensuring agreement among parties about values and predicates when necessary to avoid races, maintain safety and atomicity, or arrive at consensus
    c. Fault tolerance: Handling failures in parties or communication, including (Byzantine) misbehavior due to untrusted parties and protocols, when necessary to maintain progress or availability (See also: SF-Reliability)
    d. Tradeoffs are one focus of evaluation (See also: PDC-Evaluation)

KA Core:

5. One or more of the following mappings and mechanisms across layered systems:
    a. CPU data- and instruction-level-parallelism (See also: AR-Organization)
    b. SIMD and heterogeneous data parallelism (See also: AR-Heterogeneity)
    c. Multicore scheduled concurrency, tasks, actors (See also: OS-Scheduling)
    d. Clusters, clouds; elastic provisioning. (See also: SPD-Common)
    e. Networked distributed systems (See also: NC-Applications)
    f. Emerging technologies such as quantum computing and molecular computing

Illustrative Learning Outcomes;

CS Core:

1. Graphically show (as a Directed Acyclic Graph (DAG)) how to parallelize a compound numerical expression; for example, a = (b + c) * (d + e).
2. Explain why the concepts of consistency and fault tolerance do not arise in purely sequential programs.

KA Core:

3. Write a function that efficiently counts events such as networking packet receptions.
4. Write a filter/map/reduce program in multiple styles.
5. Write a service that creates a thread (or other procedural form of activation) to return a requested web page to each new client.

PDC-Communication: Communication

CS Core:

1. Media
    a. Varieties: channels (message passing or I/O), shared memory, heterogeneous, data stores
    b. Reliance on the availability and nature of underlying hardware, connectivity, and protocols; language support, emulation (See also: AR-IO)
2. Channels
    a. Explicit (usually named) party-to-party communication media
    b. APIs: Sockets, architectural, language-based, and toolkit constructs, such as Message Passing Interface (MPI), and layered constructs such as Remote Procedure Call (RPC) (See also: NC-Fundamentals)
    c. I/O channel APIs
3. Memory

   a. Shared memory architectures in which parties directly communicate only with memory at given addresses, with extensions to heterogeneous memory supporting multiple memory stores with explicit data transfer across them; for example, GPU local and shared memory, Direct Memory Access (DMA)
   b. Memory hierarchies: Multiple layers of sharing domains, scopes, and caches; locality: latency, false-sharing
   c. Consistency properties: Bitwise atomicity limits, coherence, local ordering
4. Data Stores
   a. Cooperatively maintained data structures implementing maps and related ADTs
   b. Varieties: Owned, shared, sharded, replicated, immutable, versioned

KA Core:

5. One or more of the following properties and extensions
   a. Topologies: Unicast, Multicast, Mailboxes, Switches; Routing via hardware and software interconnection networks
   b. Media concurrency properties: Ordering, consistency, idempotency, overlapping communication with computation
   c. Media performance: Latency, bandwidth (throughput) contention (congestion), responsiveness (liveness), reliability (error and drop rates), protocol-based progress (acks, timeouts, mediation)
   d. Media security properties: integrity, privacy, authentication, authorization (See also: SEC-Secure Coding)
   e. Data formats: Marshaling, validation, encryption, compression
   f. Channel policies: Endpoints, sessions, buffering, saturation response (waiting vs dropping), rate control
   g. Multiplexing and demultiplexing many relatively slow I/O devices or parties; completion-based and scheduler-based techniques; async-await, select and polling APIs
   h. Formalization and analysis of channel communication; for example, CSP
   i. Applications of queuing theory to model and predict performance.
   j. Memory models: sequential and release/acquire consistency
   k. Memory management; including reclamation of shared data; reference counts and alternatives
   l. Bulk data placement and transfer; reducing message traffic and improving locality; overlapping data transfer and computation; impact of data layout such as array-of-structs vs struct-of-arrays
   m. Emulating shared memory: distributed shared memory, Remote Direct Memory Access (RDMA)
   n. Data store consistency: Atomicity, linearizability, transactionality, coherence, causal ordering, conflict resolution, eventual consistency, blockchains
   o. Faults, partitioning, and partial failures; voting; protocols such as Paxos and Raft.
   p. Design tradeoffs among consistency, availability, partition (fault) tolerance; impossibility of meeting all at once
   q. Security and trust: Byzantine failures, proof of work and alternatives

Illustrative Learning Outcomes:

CS Core:

1. Explain the similarities and differences among: (1) Party A sends a message on channel X with contents 1 received by party B (2) A sets shared variable X to 1, read by B (3) A sets “X=1’ in a distributed shared map accessed by B.

KA Core:

2. Write a program that distributes different segments of a data set to multiple workers, and collects results (for the simplest example, summing segments of an array).
3. Write a parallel program that requests data from multiple sites and summarizes them using some form of reduction.
4. Compare the performance of buffered versus unbuffered versions of a producer-consumer program.
5. Determine whether a given communication scheme provides sufficient security properties for a given usage.
6. Give an example of an ordering of accesses among concurrent activities (e.g., program with a data race) that is not sequentially consistent.
7. Give an example of a scenario in which blocking message sends can deadlock.
8. Describe at least one design technique for avoiding liveness failures in programs using multiple locks.
9. Write a program that illustrates memory-access or message reordering.
10. Describe the relative merits of optimistic versus conservative concurrency control under different rates of contention among updates.
11. Give an example of a scenario in which an attempted optimistic update may never complete.
12. Modify a concurrent system to use a more scalable, reliable, or available data store.
13. Using an existing platform supporting replicated data stores, write a program that maintains a key-value mapping even when one or more hosts fail.

PDC-Coordination: Coordination

CS Core:

1. Dependencies
    a. Initiation or progress of one activity may be dependent on other activities, so as to avoid race conditions, ensure termination, or meet other requirements
    b. Ensuring progress by avoiding dependency cycles, using monotonic conditions, removing inessential dependencies
2. Control constructs and design patterns
    a. Completion-based: Barriers, joins, including termination control
    b. Data-enabled: Queues, producer-consumer designs
    c. Condition-based: Polling, retrying, backoffs, helping, suspension, signaling, timeouts
    d. Reactive: Enabling and triggering continuations
3. Atomicity
    a. Atomic instructions, enforced local access orderings
    b. Locks and mutual exclusion; lock granularity
    c. Using locks in a specific language; maintaining liveness without introducing races

d. Deadlock avoidance: Ordering, coarsening, randomized retries; backoffs, encapsulation via lock managers
e. Common errors: Failing to lock or unlock when necessary, holding locks while invoking unknown operations
f. Avoiding locks: replication, read-only, ownership, and non-blocking constructions

KA Core:

4. One or more of the following properties and extensions
    a. Progress properties including lock-free, wait-free, fairness, priority scheduling, interactions with consistency, reliability
    b. Performance with respect to contention, granularity, convoying, scaling
    c. Non-blocking data structures and algorithms
    d. Ownership and resource control
    e. Lock variants and alternatives: sequence locks, read-write locks; Read-Copy-Update (RCU), reentrancy; tickets; controlling spinning versus blocking
    f. Transaction-based control: Optimistic and conservative
    g. Distributed locking: reliability
    h. Alternatives to barriers: Clocks; counters, virtual clocks; dataflow and continuations; futures and RPC; consensus-based, gathering results with reducers and collectors
    i. Speculation, selection, cancellation; observability and security consequences
    j. Resource control using semaphores and condition variables
    k. Control flow: Scheduling computations, series-parallel loops with (possibly elected) leaders, pipelines and streams, nested parallelism
    l. Exceptions and failures. Handlers, detection, timeouts, fault tolerance, voting

Illustrative Learning Outcomes:

CS Core:

1. Show how to avoid or repair a race error in a given program.
2. Show how to ensure that a program correctly terminates when all of a set of concurrent tasks have completed.

KA Core:

3. Write a function that efficiently counts events such as sensor inputs or networking packet receptions.
4. Write a filter/map/reduce program in multiple styles.
5. Write a program in which the termination of one set of parallel actions is followed by another.
6. Write a program that speculatively searches for a solution by multiple activities, terminating others when one is found.
7. Write a program in which a numerical exception (such as divide by zero) in one activity causes termination of others.
8. Write a program for multiple parties to agree upon the current time of day; discuss its limitations compared to protocols such as network transfer protocol (NTP).
9. Write a service that creates a thread (or other procedural form of activation) to return a requested web page to each new client.

PDC-Evaluation: Evaluation

CS Core:

1. Safety and liveness requirements in terms of temporal logic constructs to express “always” and “eventually” (See also: FPL-Parallel)
2. Identifying, testing for, and repairing violations, including common forms of errors such as failure to ensure necessary ordering (race errors), atomicity (including check-then-act errors), and termination (livelock)
3. Performance requirements metrics for throughput, responsiveness, latency, availability, energy consumption, scalability, resource usage, communication costs, waiting and rate control, fairness; service level agreements (See also: SF-Performance)
4. Performance impact of design and implementation choices, including granularity, overhead, consensus costs, and energy consumption (See also: SEP-Sustainability)
5. Estimating scalability limitations, for example using Amdahl’s Law or Universal Scalability Law (See also: SF-Evaluation)

KA Core:

6. One or more of the following methods and tools:
    a. Extensions to formal sequential requirements such as linearizability
    b. Protocol, session, and transactional specifications
    c. Use of tools such as Unified Modelling Language (UML), Temporal Logic of Actions (TLA), program logics
    d. Security analysis: safety and liveness in the presence of hostile or buggy behaviors by other parties; required properties of communication mechanisms (for example lack of cross-layer leakage), input screening, rate limiting (See also: SEC-Foundations)
    e. Static analysis applied to correctness, throughput, latency, resources, energy (See also: SEP-Sustainability)
    f. Directed Acyclic Graph (DAG) model analysis of algorithmic efficiency (work, span, critical paths)
    g. Testing and debugging; tools such as race detectors, fuzzers, lock dependency checkers, unit/stress/torture tests, visualizations, continuous integration, continuous deployment, and test generators
    h. Measuring and comparing throughput, overhead, waiting, contention, communication, data movement, locality, resource usage, behavior in the presence of excessive numbers of events, clients, or threads (See also: SF-Evaluation)
    i. Application domain specific analyses and evaluation techniques

Illustrative Learning Outcomes:

CS Core:

1. Revise a specification to enable parallelism and distribution without violating other essential properties or features.
2. Explain how concurrent notions of safety and liveness extend their sequential counterparts.
3. Specify a set of invariants that must hold at each bulk-parallel step of a computation.

4. Write a test program that can reveal a data race error; for example, missing an update when two activities both try to increment a variable.
5. In a given context, explain the extent to which introducing parallelism in an otherwise sequential program would be expected to improve throughput and/or reduce latency, and how it may impact energy efficiency.
6. Show how scaling and efficiency change for sample problems without and with the assumption of problem size changing with the number of processors; further explain whether and how scalability would change under relaxations of sequential dependencies.

KA Core:

7. Specify and measure behavior when a service is requested by unexpectedly many clients.
8. Identify and repair a performance problem due to sequential bottlenecks.
9. Empirically compare throughput of two implementations of a common design (perhaps using an existing test harness framework).
10. Identify and repair a performance problem due to communication or data latency.
11. Identify and repair a performance problem due to communication or data latency.
12. Identify and repair a performance problem due to resource management overhead.
13. Identify and repair a reliability or availability problem.

PDC-Algorithms: Algorithms

CS Core:

1. Expressing and implementing algorithms in given languages and frameworks, to initiate activities (for example threads), use shared memory constructs, and channel, socket, and/or remote procedure call APIs. (See also: FPL-Parallel).
    a. Data parallel examples including map/reduce.
    b. Using channel, socket, and/or RPC APIs in a given language, with program control for sending (usually procedural) vs receiving. (usually reactive or RPC-based).
    c. Using locks, barriers, and/or synchronizers to maintain liveness without introducing races.
2. Survey of common application domains across multicore, reactive, data parallel, cluster, cloud, open distributed systems, and frameworks (with reference to the following table).

Category	**Typical Execution agents**	**Typical Communication mechanisms**	**Typical Algorithmic domains**	**Typical Engineering goals**
Multicore	Threads	Shared memory, Atomics, locks	Resource management, data processing	Throughput, latency, energy
Reactive	Handlers, threads	I/O Channels	Services, real-time	Latency
Data parallel	GPU, SIMD,	Heterogeneous	Linear algebra,	Throughput,

	accelerators, hybrid	memory	graphics, data analysis	energy
Cluster	Managed hosts	Sockets, channels	Simulation, data analysis	Throughput
Cloud	Provisioned hosts	Service APIs	Web applications	Scalability
Open distributed	Autonomous hosts	Sockets, Data stores	Fault tolerant data stores and services	Reliability

KA Core:

3. One of more of the following algorithmic domains. (See also: AL-Strategies):
    a. Linear algebra: Vector and matrix operations, numerical precision/stability, applications in data analytics and machine learning.
    b. Data processing: sorting, searching and retrieval, concurrent data structures.
    c. Graphs, search, and combinatorics: Marking, edge-parallelization, bounding, speculation, network-based analytics.
    d. Modeling and simulation: differential equations; randomization, N-body problems, genetic algorithms.
    e. Computational logic: satisfiability (SAT), concurrent logic programming.
    f. Graphics and computational geometry: Transforms, rendering, ray-tracing.
    g. Resource management: Allocating, placing, recycling and scheduling processors, memory, channels, and hosts; exclusive vs shared resources; static, dynamic and elastic algorithms; Real-time constraints; Batching, prioritization, partitioning; decentralization via work-stealing and related techniques.
    h. Services: Implementing web APIs, electronic currency, transaction systems, multiplayer games.

Illustrative Learning Outcomes:

CS Core:

1. Implement a parallel/distributed component based on a known algorithm.
2. Write a data-parallel program that for example computes the average of an array of numbers.
3. Write a producer-consumer program in which one component generates numbers, and another computes their average. Measure speedups when the numbers are small scalars versus large multi-precision values.
4. Extend an event-driven sequential program by establishing a new activity in an event handler (for example a new thread in a GUI action handler).
5. Improve the performance of a sequential component by introducing parallelism and/or distribution.
6. Choose among different parallel/distributed designs for components of a given system.

KA Core:

7. Design, implement, analyze, and evaluate a component or application for X operating in a given context, where X is in one of the listed domains, for example, a genetic algorithm for factory floor design.
8. Critique the design and implementation of an existing component or application, or one developed by classmates.
9. Compare the performance and energy efficiency of multiple implementations of a similar design, for example, multicore versus clustered versus GPU.

Professional Dispositions

- **Meticulous**: Students' attention to detail is essential when applying constructs with non-obvious correctness conditions.
- **Persistent**: Students must be prepared to try alternative approaches when solutions are not self-evident.

Mathematics Requirements

Required:
- MSF-Discrete – Logic, discrete structures including directed graphs.

Desired:
- MSF-Linear
- MSF-Calculus – Differential equations

Course Packaging Suggestions

The CS Core requirements need not be provided by a single course. They may be included across courses primarily devoted to software development, programming languages, systems, data management, networking, computer architecture, and/or algorithms.

Alternatively, the CS Core provides a basis for courses focusing on parallel and/or distributed computing. At one extreme, it is possible to offer a single broadly constructed course covering all PDC KA Core topics to varying depths. At the other extreme, it is possible to infuse PDC KA Core coverage across the curriculum with courses that cover parallel and distributed approaches alongside sequential ones for nearly every topic in computing. More conventional choices include courses that focus on one or a few categories (such as multicore or cluster), and algorithmic domains (such as linear algebra, or resource management). Such courses may go into further depth than listed in one or more KUs, and include additional software development experience, but include only CS-Core-level coverage of other topics.

As an example, a course mainly focusing on multicores could extend CS Core topics as follows.
1. Programs: KA Core on threads, tasks, instruction-level parallelism.
2. Communication: KA Core on multicore architectures, memory, concurrent data stores.